late

Machine Language

Now we will understand the binary representation of ~~number~~ instructions bcz computers only understand 1's & 0's.

In design we prefer simplicity, thus all instructions are 32 bit instruction.
meaning each instruction that we use is only 32 bit e.g add x2, x1, x0
(32 bit.)

Now there are 6 main types of instructions

1) R-type
2) I-type
3) S-type
4) B-type
5) J-type
6) U-type

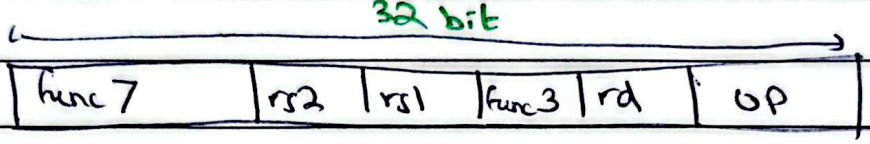
BHATTI

CamScanner

ay/date

Few key fields:

To make the 32 bit instruction:



& immediate

There are the fields that make up the 32 bits:

OPcode (Bit 6 - 0): 7 bit operation code that specifies the general instruction category.

rd (bits 11 - 7): 5-bit destination register.

funct 3 (bits 14 - 12): 3-bit functional code used to select specific operation within an opcode category.

rs1 & rs2 (bits 24-20, 19 - 15): Each 5-bit source register

funct 7 (bits 31 - 25): used like the funct 3 to select specific operations.

BHATTI

CamScanner

y/date

imm: immediate values embedded directly into the instructions.

Now it is not necessary that all these fields exist for each type of instruction.

Each instruction uses some of these fields to make its 32 bits.

But generally to revise the general format will be same.

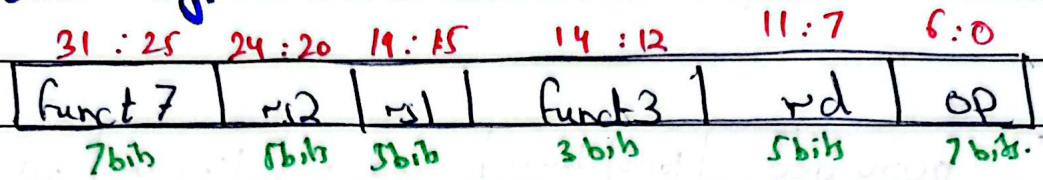
* we have 5 bits for register bcz we have (x_0-x_{31}) total 32 registers & 2^5=32, so 5 bits enough to find register.

BHATTI

CamScanner

date

1) R-type:

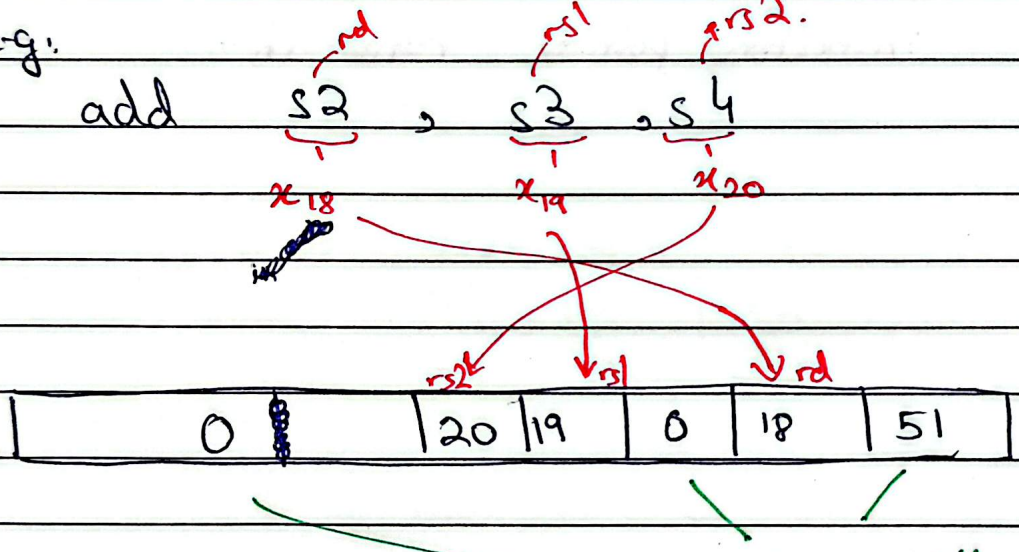


- Register – Register operations.

e.g add, sub, mul, div, shl, slb etc.

e.g.:

add s2 , s3 , s4



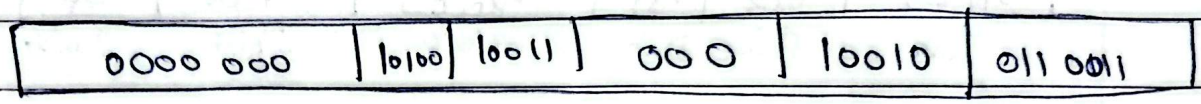
There will be given in some table but for add it's opcode is 51.

now see even though we write s2, at backend it means x18 & thus 18 is saved in rd.

BHATTI

'date

also in computer decimal not used instead:



or in Hex:

0x01498933.

This is how instruction code is written at memory & runs when instruction pointer runs it.

BHATTI

day/date

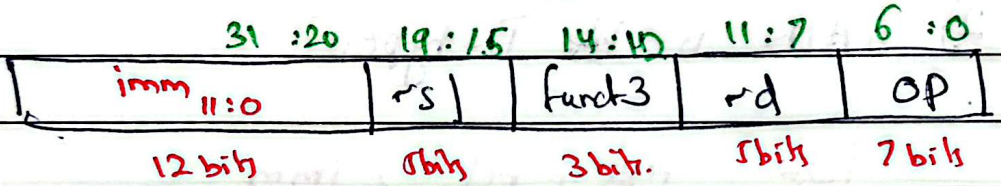
2) I-type:

immediate type operations & loading memories to registers.

In this our 32 bit needs to store ~~memory~~ immediate ~~bit~~ & also there is no rs2.

3 main operands:

- rs1
- rd
- imm



now immediate takes place of both rs2 + funct7 = 5+7 = 12 bits.

eg: addi, lw, andi, slli, jalr

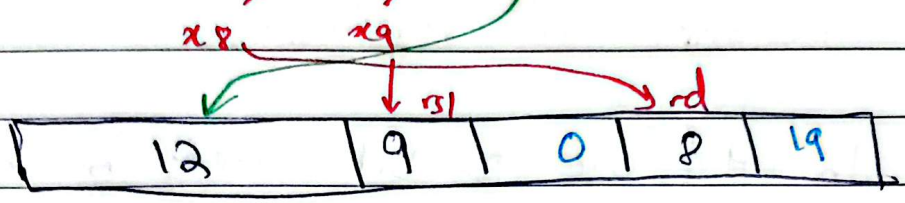
Imp: jalr is also I-type & not J-type, bcz it needs rs1.

BHATTI

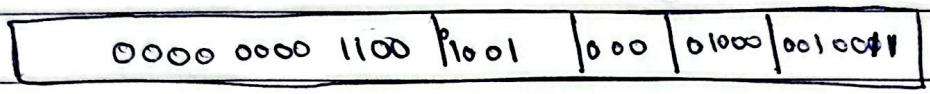
day/date

eg.

1) addi s0 , s1 , 12



or



or

0x00C48413

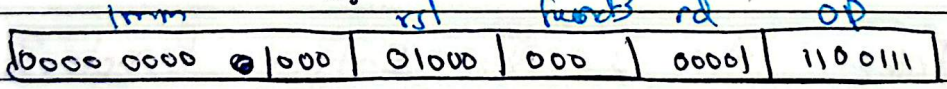
2) JALR is I-type:

jalr rd , rs1 , imm

bcz jalr need all these 3 it is classified as I-type & its imm is max 12 bits.

eg jalr ra , s0 , 8

x8



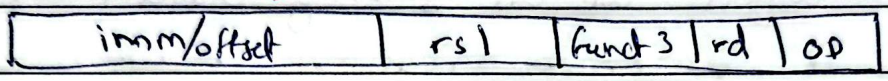
0x008400E7

BHATTI

late

3) lw is lb, lh are also i-type

lw rd, offset(rs1)



it follows exact same code of an i-type so it is i-type.

BHATTI

CamScanner

date

3) Store - type: (S-type):

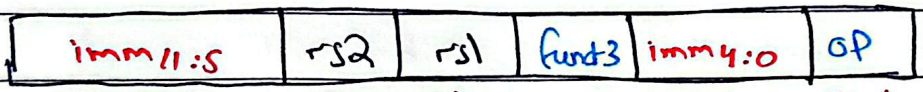
memory store operations

when we are storing data, we don't have a destination register, we instead have 2 source register.

`sw rs2, offset(rs1)`

Remember rs1 & rs2 read whereas rd only instruction

now to make the 32 bit instruction, our immediate will take place of rd & funct7.

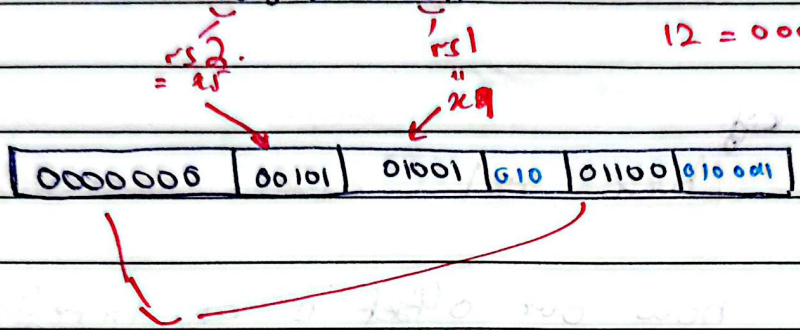


now our immediate is divided across the 32 bits & not in one place because we had to give the place of rd to immediate. We do this so that no new format has to be made when registers are pushed to another place, so design remain simple & similar

BHATTI

e.g:

sw t0, 12(s1)



BHATTI

CamScanner

day/date

4) B-type: (Branch type):

for conditional branches

bxx rs1 , rs2 , address offset

now our offset is an immediate, so it looks just like S-type.
Yes its format will be like S-type except there is a small change.

The immediate is actually 13 bits. BUT HOW?
In RISC-V all instructions are either 2 bytes or 4-byte aligned & branch targets are always even addresses.
Thus bit 0 of our immediate is always 0. so we do not store it but implicitly consider it.

imm_{13} = 0 · 0 0000 00 0000 0

(0 · 0 0000 00 0000 : These 12 bits are actually stored; 0 : This is implicitly considered)

BHATTI

date

Also offset is only 13 bits, then how is able to store any address while we know our addresses are 32-bit.

This is because it does not store address, it stores the offset.

Offset = TARget address − B-type instruction current address (PC)

so when we branch at back:

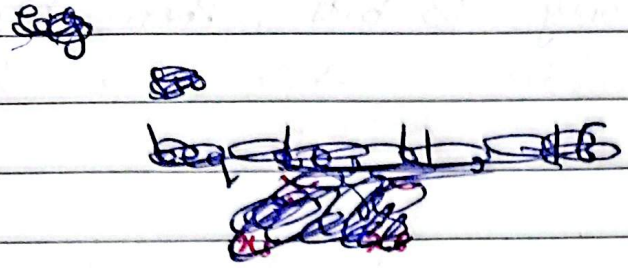
Target address = PC + offset,
is calculated to find the address

This is why we cannot Jump more then 13-bit distances & Jumps are capped there.
Because our difference can max be 13-bit.

BHATTI

CamScanner

te



imm 12,10:5	rs2	rs1	funct3	imm 4:1,11	Opcode

immediate address is quite strange becaus our 11 bit of immediate ß in 7 bit of 32 bit instruction

e.g

01 00100	rs2	rs1	---	01011	---

now our immediate is:

→ 0 1 | 00100 | 0101 | 0

implicit bit 0.

we will do example at end of this topic

BHATTI

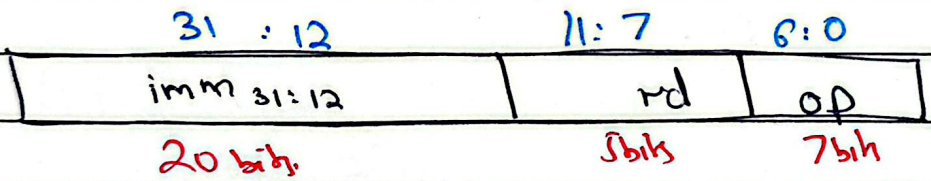
5) U - TYPE

"upper immediate type"

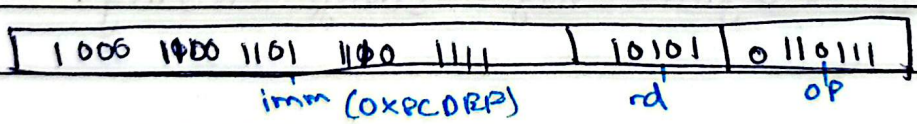
used for "lui" command.

lui rd, 20 bit immediate.

This is quite straight forward: rs1 & rs2 both are not used here. There is only one lui so no need of funct3 or funct7 as well so total bits saved = 5+5+3+7 = 20.



e.g. lui x5, 0x8CDEF



0x8CDEFAB7.

BHATTI

6) J-type

"Unconditional Jumps."

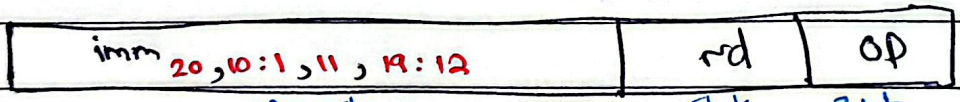
used by Jal.

not by jalr that is I-type

jal rd, offset.

now jal is also like U-type but in this we can actually store 21 Bits. How?

exactly same way as we did in B-type due to alignment in even numbers, our 0 bit is always 0.



* again very strange encoding.

BHATTI

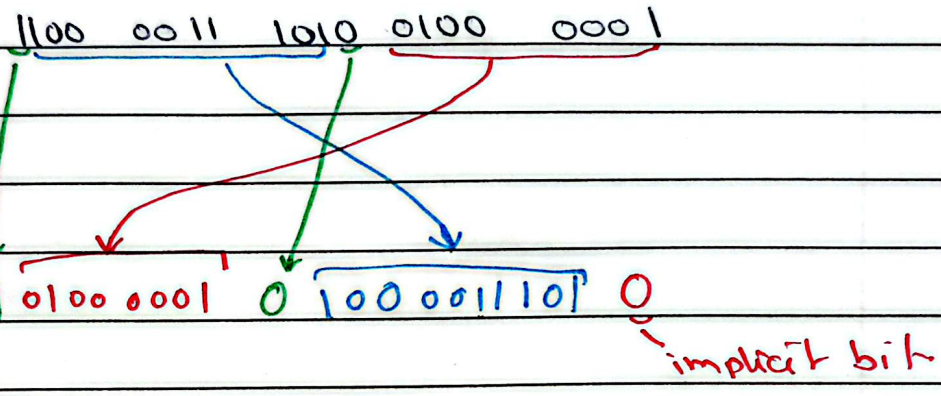
CamScanner

te

e.g

1100 0011 1010 0100 0001	rd	op

if we actually convert this immediate back to decimal:



so actual number = 0 x 14143A = 1315898

- offset:

Again here offset means the difference in PC & target address.

Target address = PC + offset.

BHATTI

CamScanner

All formats:

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
funct7	rs2	rs1	funct3	rd	op	**R-Type**
$imm_{11:0}$		rs1	funct3	rd	op	**I-Type**
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	**S-Type**
$imm_{12,10:5}$	rs2	rs1	funct3	$imm_{4:1,11}$	op	**B-Type**
$imm_{31:12}$				rd	op	**U-Type**
$imm_{20,10:1,11,19:12}$				rd	op	**J-Type**
20 bits				5 bits	7 bits	

CamScanner

y/date

Few common instruction fields:

Instruction Fields & Formats

Instruction	op	funct3	Funct7	Type
add	0110011 (51)	000 (0)	0000000 (0)	R-Type
sub	0110011 (51)	000 (0)	0100000 (32)	R-Type
and	0110011 (51)	111 (7)	0000000 (0)	R-Type
or	0110011 (51)	110 (6)	0000000 (0)	R-Type
addi	0010011 (19)	000 (0)	-	I-Type
beq	1100011 (99)	000 (0)	-	B-Type
bne	1100011 (99)	001 (1)	-	B-Type
lw	0000011 (3)	010 (2)	-	I-Type
sw	0100011 (35)	010 (2)	-	S-Type
jal	1101111 (111)	-	-	J-Type
jalr	1100111 (103)	000 (0)	-	I-Type
lui	0110111 (55)	-	-	U-Type

CamScanner

day/date

Addressing Modes:

There are also 4 addressing modes for operands.

1) Register only
2) Immediate
3) Base addressing
4) PC-Relative.

1) Register Only:

Operands are register only.

e.g add , sub , mul.

typically use R-type format.

2) immediate Only:

"operands directly inside instructions:

e.g addi , ori , lui

typically use I-type format

BHATTI

CamScanner

date

3) Base addressing:

load & stores.

Address = Register + immediate offset.

e.g sw & lw ; I-type.

S-type & I-type (lw).

4) PC relative:

Address = PC + immediate offset

we have seen this in 2 formats:

B-type & J-type.

e.g bne, jal. (not jalr that is base).

BHATTI

CamScanner

ay/date

Examples:

Q1) convert following RISC-V assembly to machine code in hexadecimal.

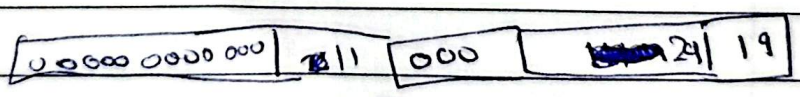
Instruction address		instruction
0x000A0028	Fund1:	addi t4, a1, 0
0x--A002C		ori a0, a0, 32
0x--A0030		sub a1, a1, a0
0x--A0034		jal Func2

0x-A00058	Fund2:	lw t2, 4(a0)
0x-A005C		sw t2, 16(a1)
0x-A0060		srli t3, t2, 8
0x--A0064		beq t2, t3, Else
0x-A0068		jr ra
0xA006C		addi a0, a0, 4
0x--A0070		j Func2.

Solution

addi t4, a1, 0:

x29 x11



in Hex: 0x00058E93.

BHATTI

CamScanner

y/date

ori a0, a0, 32:

0000 0010 0000	01010	110	01010	0010011

0x02056513.

sub a1, a1, a0:

0x 40 A585B3

jal Func2: (J-type). | immediate | rd | op |

First we need to calculate Offset.

Offset = Target − PC

= 0xA0058 − 0xA00384

= 0x000204

= 0000 0000 0000 0010 0100

But imm is arranged like (20, 10:1, 11, 19:12)

So

imm_{20} = 0 0000 0100 0000 0000 0 (implied)

= 0x02400

So complete: | 0x02400 | rd | 1101111 |

day/date

$= 0 \times 024000EF$

Like this do the rest of need.

BHATTI

CamScanner